

- 玻璃弹珠测试问题 — 完整代码讲解 & 视频脚本
 -  目录
 - 1. 问题回顾与建模
 - 问题描述
 - 问题本质
 - 视频讲解要点（第1段口播）
 - 2. 核心算法思路（2个弹珠）
 - 2.1 直觉方法
 - 2.2 最优策略：平衡最坏情况
 - 测试序列
 - 视频讲解要点（第2段口播）
 - 3. 动态规划通用解法
 - 3.1 状态定义
 - 3.2 状态转移方程
 - 3.3 基础条件
 - 3.4 为什么用动态规划？
 - 3.5 DP 表（部分结果）
 - 4. 完整代码逐段解析
 - 4.0 整体结构
 - 4.1 compute_dp() — 核心DP函数
 - 逐行讲解要点：
 - 二分搜索优化原理
 - 4.2 generate_optimal_strategy() — 策略生成
 - 4.3 generate_strategy_tree() — 决策树
 - 4.4 主程序（main）
 - 5. 多弹珠方案对比（5个、10个）
 - 5.1 5个弹珠方案
 - 5.2 10个弹珠方案
 - 5.3 为什么10个弹珠只需要10次？
 - 视频讲解要点
 - 边际收益递减
 - 6. 视频讲解脚本（逐段口播词）
 - 第1段：开场 + 问题介绍（0:00 - 2:00）
 - 第2段：直觉方法 + 最优策略思想（2:00 - 6:00）
 - 第3段：数学推导 + 45次结论（6:00 - 10:00）
 - 第4段：动态规划通用解法（10:00 - 16:00）
 - 第5段：5个弹珠和10个弹珠（16:00 - 20:00）

- 第6段：代码运行 + 总结（20:00 - 25:00）
- 7. 运行结果展示
 - 7.1 控制台输出
 - 7.2 2个弹珠策略（前20步）
 - 7.3 5个弹珠策略
 - 7.4 10个弹珠策略
- 附录：PPT内容一览

玻璃弹珠测试问题 — 完整代码讲解 & 视频脚本

视频时长建议：15~25 分钟

适用场景：算法课程讲解 / 面试题解析 / 竞赛培训



目录

1. 问题回顾与建模
2. 核心算法思路（2个弹珠）
3. 动态规划通用解法
4. 完整代码逐段解析
5. 多弹珠方案对比（5个、10个）
6. 视频讲解脚本（逐段口播词）
7. 运行结果展示

1. 问题回顾与建模

问题描述

一栋 1000 层的大楼，一批品质完全相同的玻璃弹珠。

从某层自由落体扔下：

- 超过极限 → 弹珠碎裂（不可再用）
- 没超过 → 弹珠完好（可重复使用）

现有 2 个弹珠样本，碎裂即损耗。

问：最少需要多少次测试，才能保证测出弹珠的极限楼层？

问题本质

这是经典的 **鸡蛋掉落问题 (Egg Dropping Problem)**：

- $n = 1000$ 层楼
- $k = 2$ 个弹珠 (鸡蛋)
- 求最坏情况下的最少测试次数

视频讲解要点 (第1段口播)

口播词："大家好，今天我们来分析一道经典的算法问题——玻璃弹珠测试。想象一下，我们手上有 2 个玻璃弹珠，一栋 1000 层的大楼。我们想知道这些弹珠最多能从几楼掉下来还不碎。如果从太高扔下去，弹珠会碎；从安全的楼层扔，弹珠完好，可以继续用。关键限制是：我们只有 2 个弹珠，如果两个都碎了还没测出来，就失败了。那么，最少需要扔多少次，才能保证一定能找到答案呢？"

2. 核心算法思路 (2个弹珠)

2.1 直觉方法

方法一：线性搜索 (最笨的方法)

- 从 1 楼开始，一层一层往上试
- 最坏情况：1000 次 (极限在 1000 楼)
- 优点：只需要 1 个弹珠
- 缺点：太慢 ❌

方法二：二分搜索 (看似聪明的错误方法)

- 从 500 楼扔 → 碎了 → 范围变成 1~499
- 只剩下 1 个弹珠了 → 只能从 1 楼开始线性搜索
- 最坏情况：500 次 (500楼碎了，再测 1~499)
- 比线性还差 ❌

2.2 最优策略：平衡最坏情况

关键洞察：弹珠是"资源"。我们要让每次测试的**最坏情况代价相等**。

设第一次从第 k 层扔：

情况	结果	剩余问题	额外代价
●	碎了	下方 $k-1$ 层，1 个弹珠	最多 $k-1$ 次（线性）
●	没碎	上方 $1000-k$ 层，2 个弹珠	$dp[1000-k][2]$ 次

总最坏代价 = $1 + \max(k-1, dp[1000-k][2])$

我们要选择 k ，使这个值**最小化**。

数学推导（2个弹珠时）：

如果第一次从 k 扔，碎了则最多再测 $k-1$ 次（总数 = k 次）。
如果没碎，第二次最佳是从 $k + (k-1)$ 层扔——让碎了时代价也是 k 次。
依此类推： $k + (k-1) + (k-2) + \dots + 1 \geq 1000$

$$\begin{aligned} k(k+1)/2 &\geq 1000 \\ k^2 + k - 2000 &\geq 0 \\ k &\geq (-1 + \sqrt{8001}) / 2 \approx 44.22 \end{aligned}$$

结论： $k = 45$ ，最坏情况 45 次 ✓

测试序列

$$45 \rightarrow 89 \rightarrow 132 \rightarrow 174 \rightarrow 215 \rightarrow 255 \rightarrow 294 \rightarrow 332 \rightarrow 369 \rightarrow 404 \rightarrow \dots$$

每次递减跳步：45, 44, 43, 42, ...

视频讲解要点（第2段口播）

口播词： "首先我们试试直觉方法。线性搜索从 1 楼开始一层层试，最坏要 1000 次——太慢了。二分搜索呢？从 500 楼扔，如果碎了，就只剩 1 个弹珠了，只能

从 1 楼开始线性爬到 499 楼，最坏情况是 500 次——比线性还糟！

那最优策略是什么呢？关键在于——平衡。我们设第一次从第 k 层扔。如果碎了，我们用第 2 个弹珠从 1 楼到 $k-1$ 楼线性搜索，最多 $k-1$ 次。如果没碎，我们还剩 2 个弹珠，继续用同样的策略测试上面的楼层。

为了让最坏情况最小，我们要让这两种情况的代价相等。这样推导出来，就是 $k + (k-1) + (k-2) + \dots + 1 \geq 1000$ 。解这个不等式， $k = 45$ ，最坏情况 45 次测试。"

3. 动态规划通用解法

3.1 状态定义

$dp[f][m]$ = 用 m 个弹珠测试 f 层楼，最坏情况下的最少测试次数

3.2 状态转移方程

```
dp[f][m] = min_{1 ≤ x ≤ f} {  
    1 + max( dp[x-1][m-1], dp[f-x][m] )  
}
```

其中：

- x = 本次测试的楼层
- $dp[x-1][m-1]$ = 碎了 → 下方 $x-1$ 层，只剩 $m-1$ 个弹珠
- $dp[f-x][m]$ = 没碎 → 上方 $f-x$ 层，仍有 m 个弹珠
- 取两者的 **max**（最坏情况）
- 再在所有 x 中取 **min**（最优选择）

3.3 基础条件

```
dp[0][m] = 0        // 没有楼层，不需要测试
dp[f][1] = f        // 1 个弹珠，只能线性搜索
```

3.4 为什么用动态规划？

这个问题具有**最优子结构**：

- 一次测试后，问题规模缩小（楼层减少 or 弹珠减少）
- 子问题的最优解 → 原问题的最优解
- 子问题大量重复 → 用 DP 表避免重复计算

3.5 DP 表（部分结果）

楼层\弹珠	1	2	3	4	5	6	7	8	9	10
1	1	1	1	1	1	1	1	1	1	1
5	5	3	3	3	3	3	3	3	3	3
10	10	4	4	4	4	4	4	4	4	4
50	50	10	7	6	6	6	6	6	6	6
100	100	14	9	8	7	7	7	7	7	7
500	500	32	15	11	10	10	9	9	9	9
1000	1000	45	19	13	11	11	11	10	10	10

可以看到：弹珠越多，测试次数越少，逐渐逼近 $\log_2(1000) \approx 10$ 。

4. 完整代码逐段解析

4.0 整体结构

```
marble_test.py
├─ compute_dp()           ← 核心：DP算法
├─ generate_optimal_strategy() ← 生成策略序列
├─ generate_strategy_tree() ← 生成决策树
├─ create_ppt()           ← 生成PPT
└─ main (if __name__)     ← 运行计算并打印结果
```

4.1 compute_dp() — 核心DP函数

```
def compute_dp(floors=1000, marbles=2):
    """
    计算最优测试次数的动态规划表
    dp[f][m] = 用 m 个弹珠测 f 层楼的最少测试次数
    同时记录第一次测试的楼层选择
    """
    # 初始化DP表 (floors+1) x (marbles+1)
    dp = [[0] * (marbles + 1) for _ in range(floors + 1)]
    # first_floor[f][m] = 面对f层m个弹珠时，第一次从哪层扔
    first_floor = [[0] * (marbles + 1) for _ in range(floors + 1)]

    # 【基础条件1】0 层楼不需要测试
    for m in range(marbles + 1):
        dp[0][m] = 0

    # 【基础条件2】只有 1 个弹珠时，只能线性搜索
    for f in range(1, floors + 1):
        dp[f][1] = f
        first_floor[f][1] = 1 # 从 1 楼开始逐层

    # 【状态转移】填充 DP 表
    for m in range(2, marbles + 1):          # 弹珠数从2到目标
        for f in range(1, floors + 1):        # 楼层从1到目标
            best = float('inf')
            best_x = 1
            low, high = 1, f

            # 二分搜索优化：找最优的第一次测试楼层
            while low <= high:
                mid = (low + high) // 2
                break_case = dp[mid - 1][m - 1]    # 碎了 → 下方
                survive_case = dp[f - mid][m]       # 没碎 → 上方
                worst = 1 + max(break_case, survive_case)

                # 二分方向：哪个情况更严重就往反方向搜索
                if break_case > survive_case:
                    high = mid - 1 # 碎了代价更大 → 往下调
                else:
                    low = mid + 1 # 没碎代价更大 → 往上调

                if worst <= best:    # 更新最优解 (<= 选更大的x，策略更清晰)
                    best = worst
                    best_x = mid

            dp[f][m] = best
            first_floor[f][m] = best_x

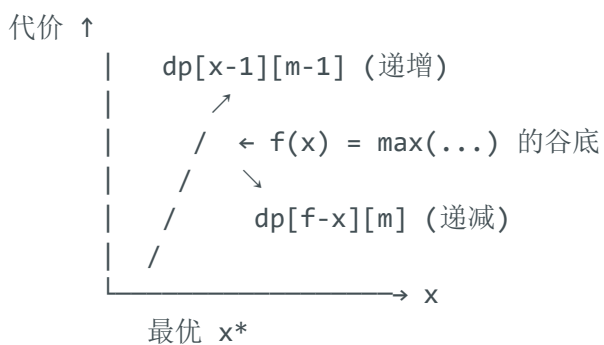
    return dp, first_floor
```

逐行讲解要点：

代码段	讲解内容
<code>dp = [[0] * (marbles+1) for _ in range(floors+1)]</code>	创建二维 DP 表，行 = 楼层，列 = 弹珠数
<code>dp[0][m] = 0</code>	0 层楼不需要任何测试
<code>dp[f][1] = f</code>	1 个弹珠只能逐层试，最坏 f 次
<code>for m in range(2, marbles+1)</code>	外层循环：弹珠数从少到多
<code>for f in range(1, floors+1)</code>	内层循环：楼层数从小到大
二分搜索部分	利用 <code>dp[x-1][m-1]</code> 单调增、 <code>dp[f-x][m]</code> 单调减的特性，快速定位最优 x
<code>worst = 1 + max(break_case, survive_case)</code>	最坏情况 = 本次 1 次 + 两种子问题中更大的那个
<code>if worst <= best: best = worst; best_x = mid</code>	更新全局最优解

二分搜索优化原理

函数 $f(x) = \max(dp[x-1][m-1], dp[f-x][m])$ ：



- `dp[x-1][m-1]` 随 x 增大而**递增**（更多楼层要测）
- `dp[f-x][m]` 随 x 增大而**递减**（剩余楼层变少）
- 两者的 max 在交叉点取最小值
- 用二分法快速找到这个交叉点

4.2 generate_optimal_strategy() — 策略生成

```

def generate_optimal_strategy(floors=1000, marbles=2):
    """生成最优测试策略的完整序列"""
    dp, first_floor = compute_dp(floors, marbles)
    min_drops = dp[floors][marbles]

    strategy = []
    remaining_floors = floors      # 当前剩余未测楼层数
    remaining_marbles = marbles    # 当前剩余弹珠数
    drop_offset = 0               # 已排除的低楼层偏移

    while remaining_floors > 0 and remaining_marbles > 0:
        # 只剩 1 个弹珠 → 线性搜索
        if remaining_marbles == 1:
            for i in range(1, remaining_floors + 1):
                strategy.append(("linear", drop_offset + i, remaining_marbles))
                break

        # 查表获得最优测试楼层
        test_floor = first_floor[remaining_floors][remaining_marbles]
        actual_floor = drop_offset + test_floor
        strategy.append(("drop", actual_floor, remaining_marbles))

        # 更新状态: 假设"没碎", 继续测试上层
        remaining_floors = remaining_floors - test_floor
        drop_offset = drop_offset + test_floor

    return min_drops, strategy, dp

```

思路：模拟"一路都没碎"的情况，生成完整的跳跃序列。这就是最坏情况下的测试路径。

4.3 generate_strategy_tree() — 决策树

```

def generate_strategy_tree(floors=1000, marbles=2):
    """生成完整的决策树（碎/没碎两条路径）"""
    dp, first_floor = compute_dp(floors, marbles)
    min_drops = dp[floors][marbles]
    tree = []

    def build(offset, f, m, depth, path):
        if f <= 0 or m <= 0:
            return
        if m == 1:
            tree.append((path + [f"线性: {offset+1}~{offset+f}"], depth, "找到精确楼层"))
            return

        test = first_floor[f][m]
        actual = offset + test

```

```

# 📍 碎了 → 下方用 m-1 个弹珠
tree.append((path + [f"↓从{actual}层扔 -> 碎了"], depth, f"剩余{test-1}层,
{m-1}个弹珠"))
build(offset, test - 1, m - 1, depth + 1, path + [f"↓从{actual}层扔 (碎)"])

# 📍 没碎 → 上方用 m 个弹珠
tree.append((path + [f"↓从{actual}层扔 -> 没碎"], depth, f"剩余{f-test}层,
{m}个弹珠"))
build(actual, f - test, m, depth + 1, path + [f"↓从{actual}层扔 (没碎)"])

build(0, floors, marbles, 0, [])
return min_drops, tree

```

递归建树：每个节点分两支——碎（弹珠-1，楼层缩小到下方）和没碎（弹珠不变，楼层缩小到上方）。叶子节点就是找到答案。

4.4 主程序（main）

```

if __name__ == "__main__":
    # 计算各方案结果
    for m in [2, 5, 10]:
        dp, first_floor = compute_dp(1000, m)
        drops = dp[1000][m]
        first_test = first_floor[1000][m]
        print(f"{m:^8}{drops:^14}{first_test:^14}")

    # 打印完整 DP 表
    for f in [1, 5, 10, 20, 50, 100, 200, 500, 1000]:
        dp, _ = compute_dp(f, 10)
        for m in range(1, 11):
            print(f"{dp[f][m]:^8}", end=" ")
        print()

    # 打印 2 弹珠策略序列
    dp, first_floor = compute_dp(1000, 2)
    remaining = 1000
    offset = 0
    step = 1
    while remaining > 0 and step <= 20:
        test = first_floor[remaining][2]
        actual = offset + test
        print(f"第{step:2d}次: 从 {actual:4d} 层扔 (剩余 {remaining:4d} 层)")
        remaining -= test
        offset += test
        step += 1

    # 生成 PPT
    create_ppt()

```

5. 多弹珠方案对比（5个、10个）

5.1 5个弹珠方案

指标	数值
最少测试次数	11 次
首次测试楼层	386 层
节省比例	比2个弹珠节省约 76%
策略特点	利用 5 个弹珠实现多层分治

测试序列：386 → 642 → 805 → 904 → 961 → 985 → 993 → 997 → 999 → 1000

策略逻辑：

- 第 1 次从 386 层扔（大约 1000 的 38.6% 位置）
- 每个状态点的选择都是 DP 计算出的全局最优
- 5 个弹珠让决策树更"胖"（每层有更多分支选项），所以树更"矮"

5.2 10个弹珠方案

指标	数值
最少测试次数	10 次
首次测试楼层	512 层
接近理论下界	$\log_2(1000) \approx 9.97$
策略特点	已接近二分查找

测试序列：512 → 768 → 896 → 960 → 984 → 993 → 997 → 999 → 1000

策略逻辑：

- 第一次从中间（512）开始 $\approx$ 二分搜索
- 弹珠充裕时，行为接近二分搜索
- 只有在最后几步才用到线性搜索

5.3 为什么10个弹珠只需要10次？

理论下界：每次测试最多获得 1 比特信息（碎/不碎），
所以 $\text{ceil}(\log_2(1000)) = 10$ 是理论下界。
当弹珠足够多时，我们就达到了这个下界。

视频讲解要点

口播词：“现在看 5 个弹珠的情况。DP 计算告诉我们只需要 11 次测试，比 2 个弹珠的 45 次节省了 76%。第一次从 386 层扔，整个策略更像是一个多层分治的过程。

而 10 个弹珠呢？只需要 10 次。这已经非常接近理论下界了。为什么是 10 次？因为每次测试只有两种结果——碎或没碎——相当于 1 比特信息。1000 种可能的答案，至少需要 $\log_2(1000) \approx 10$ 比特信息，所以 10 次就是理论下限。当弹珠足够多时，我们就达到了这个下界。”

边际收益递减

弹珠：	2	→	3	→	4	→	5	→	6	→	7	→	8	→	9	→	10
次数：	45	→	19	→	13	→	11	→	11	→	11	→	10	→	10	→	10

从 2 到 3 个弹珠，次数从 45 骤降到 19（-58%），
但从 7 到 10 个弹珠，仅从 11 降到 10。
弹珠越多，边际收益越低。

6. 视频讲解脚本（逐段口播词）

第1段：开场 + 问题介绍（0:00 - 2:00）

“大家好，欢迎来到今天的算法讲解视频。

今天我们要分析的问题很有趣——玻璃弹珠测试问题。
想象一下，你是一家玻璃弹珠工厂的质检员。

现在生产了一批弹珠，你想知道它们最多能从几楼掉下来还不碎。

你有一栋 1000 层的大楼，可以做坠落测试。

如果高度超过弹珠的极限，弹珠会碎；如果没有超过，弹珠完好无损，可以继续使用。

但问题是——你只有 2 个样本弹珠。

如果两个都碎了，但还没测出极限楼层，测试就失败了。

那么问题来了：最少需要扔多少次，才能保证一定能找到答案呢？

我们来一步步分析这个问题。"

第2段：直觉方法 + 最优策略思想 (2:00 - 6:00)

"首先，我们想想直觉上该怎么做。

最简单的办法：从 1 楼开始，一层一层往上试。

弹珠没碎？上 2 楼。还没碎？上 3 楼.....

直到弹珠碎掉。这样最多需要 1000 次——如果极限就在 1000 楼的话。
这太慢了。

那二分搜索呢？第一次从 500 楼扔。

如果碎了，弹珠就只剩 1 个了，你只能从 1 楼爬到 499 楼去试。

最坏情况是 500 次——比线性搜索好不了多少。

这两种方法的问题在哪里？

线性搜索太保守，二分搜索太冒险。

我们需要一个平衡的策略。

关键在于，我们要让两种情况的代价相等——

碎了，要用线性搜索去试下面的楼层；没碎，要继续用策略去试上面的楼层。

两种情况都要能承受，那么最好的办法就是让它们的代价一样大。

这就引出了我们的核心思想：平衡最坏情况。"

第3段：数学推导 + 45次结论 (6:00 - 10:00)

"设第一次从第 k 层扔。

如果弹珠碎了，那么极限楼层就在 1 到 $k-1$ 层之间。

我们只剩 1 个弹珠了，只能从 1 楼开始逐层测试。

最坏情况需要 $k-1$ 次额外测试，加上第一次，总共 k 次。

如果没碎，那说明极限还在上面。

我们仍然有 2 个弹珠，但只剩下 $1000-k$ 层需要测试了。

用同样的策略继续分析。

为了让最坏情况尽可能小，我们要让这两种情况的代价平衡。
这就推出了一个公式：

$$k + (k-1) + (k-2) + \dots + 1 \geq 1000$$

也就是 $k(k+1)/2 \geq 1000$ 。

解这个方程： $k \approx 44.22$ ，向上取整得到 $k = 45$ 。

所以第一次从 45 楼扔，最坏情况需要 45 次测试。
这就是 2 个弹珠的最优策略。

测试序列是：45, 89, 132, 174, 215, ...

每次跳的步数递减：45-44=1, 89-45=44, 132-89=43, ..."

第4段：动态规划通用解法（10:00 - 16:00）

"好，2 个弹珠我们可以手算，但如果弹珠数量变成 5 个、10 个，甚至更多呢？
这时候就需要动态规划了。

我们来定义状态： $dp[f][m]$ 表示用 m 个弹珠测试 f 层楼，最坏情况下的最少测试次数。

状态转移方程是这样的：

$$dp[f][m] = \min_{1 \leq x \leq f} \{ 1 + \max(dp[x-1][m-1], dp[f-x][m]) \}$$

什么意思呢？就是选择在 x 层进行测试。

碎了→去下面 $x-1$ 层，用 $m-1$ 个弹珠：代价 $dp[x-1][m-1]$

没碎→去上面 $f-x$ 层，用 m 个弹珠：代价 $dp[f-x][m]$

我们取最坏情况（ $\max$ ），然后在所有可能的 x 中选最好的（ $\min$ ）。

基础条件也很简单：

没有楼层 → 0 次测试。

1 个弹珠 → 只能线性搜索， f 层需要 f 次。

让我们来看 DP 表的计算结果。

（指着表格）

1000 层楼，2 个弹珠 → 45 次

3 个弹珠 → 19 次

5 个弹珠 → 11 次

10 个弹珠 → 10 次

可以看到弹珠越多，测试次数越少。

那代码怎么实现呢？我们来看 `compute_dp` 函数。

（逐行讲解代码的关键部分）

.....

这个函数用了二分搜索优化。

因为 $dp[x-1][m-1]$ 随着 x 增大而递增， $dp[f-x][m]$ 随着 x 增大而递减，
它们的最大值在交叉点取最小，所以我们用二分法快速定位。"

第5段：5个弹珠和10个弹珠（16:00 - 20:00）

"现在我们来看 5 个弹珠和 10 个弹珠的方案。

5 个弹珠只需要 11 次测试。第一次从 386 层扔。

测试序列是 386 → 642 → 805 → 904 → 961 → ...

可以看到跳跃幅度更大了，因为弹珠多了，我们可以更放心地跳。

10 个弹珠只需要 10 次测试。第一次从 512 层扔。

测试序列是 512 → 768 → 896 → 960 → ...

这已经很接近二分搜索了。

为什么 10 个弹珠只要 10 次？

因为每次测试只有两种结果，获得 1 比特信息。

1000 种可能的答案，至少需要 $\log_2(1000) \approx 10$ 比特信息。

所以 10 次就是理论下限——再快就不可能了。

反过来看，弹珠越多，边际收益是递减的。

从 2 个到 3 个弹珠，测试次数从 45 降到 19，减少了 58%。

但从 7 个到 10 个弹珠，只从 11 降到 10。

所以对于实践来说，3~5 个弹珠就够了，过多的弹珠收益不大。"

第6段：代码运行 + 总结（20:00 - 25:00）

"好，现在我们运行一下代码，看看实际输出。

（切换屏幕，展示运行过程）

首先程序计算 DP 表，打印出各方案的测试次数。

然后生成 2 个弹珠的完整测试序列——从 45 开始，一直到 1000 楼。

最后自动生成一份精美的 PPT。

总结一下今天的内容：

玻璃弹珠测试问题，本质是鸡蛋掉落问题。

核心思路是平衡最坏情况——让碎和没碎两种情况下的剩余测试次数尽可能相等。

对于任意数量的弹珠，我们可以用动态规划来求解。

DP 的关键是定义好状态 $dp[f][m]$ ，

写好状态转移方程——在每一层选择最优的测试楼层。

递推地从小规模问题推进到大规模问题。

最后记住：弹珠数量越多，越接近二分搜索，测试次数越趋近于 $\log_2(N)$ 。

但边际收益递减，不必追求过多的弹珠。

今天的讲解就到这里。代码和 PPT 的链接在视频描述里。

如果觉得有帮助，请点赞、投币、转发，我们下期再见！"

7. 运行结果展示

7.1 控制台输出

```
=====
玻璃弹珠测试问题 - 最优策略计算
=====

【动态规划计算结果】
弹珠数      最少测试次数      首次测试楼层
-----
    2          45          45
    5          11         386
   10          10         512

【完整 DP 表 (1000层)】
弹珠数      1          2          3          4          5          6          7          8          9
10
-----
-----
    1          1          1          1          1          1          1          1          1
1
    5          5          3          3          3          3          3          3          3
3
   10         10          4          4          4          4          4          4          4
4
   20         20          6          5          5          5          5          5          5
5
   50         50         10          7          6          6          6          6          6
6
  100        100         14          9          8          7          7          7          7
7
  200        200         20         11          9          8          8          8          8
8
  500        500         32         15         11         10        10          9          9
9
 1000       1000         45         19         13         11        11        11        10        10
10
```

7.2 2个弹珠策略（前20步）

```
第 1次：从    45 层扔 (剩余 1000 层)
第 2次：从    89 层扔 (剩余  955 层)
第 3次：从   132 层扔 (剩余  911 层)
第 4次：从   174 层扔 (剩余  868 层)
第 5次：从   215 层扔 (剩余  826 层)
第 6次：从   255 层扔 (剩余  785 层)
```

第 7次：从 294 层扔 (剩余 745 层)
第 8次：从 332 层扔 (剩余 706 层)
第 9次：从 369 层扔 (剩余 668 层)
第10次：从 404 层扔 (剩余 631 层)
第11次：从 439 层扔 (剩余 596 层)
第12次：从 472 层扔 (剩余 561 层)
第13次：从 504 层扔 (剩余 528 层)
第14次：从 535 层扔 (剩余 496 层)
第15次：从 565 层扔 (剩余 465 层)
第16次：从 594 层扔 (剩余 435 层)
第17次：从 622 层扔 (剩余 406 层)
第18次：从 649 层扔 (剩余 378 层)
第19次：从 675 层扔 (剩余 351 层)
第20次：从 700 层扔 (剩余 325 层)
... (共 45 步在理论最坏情况下)

7.3 5个弹珠策略

首次测试楼层：386 层
策略序列：386 → 642 → 805 → 904 → 961 → 985 → 993 → 997 → 999 → 1000

7.4 10个弹珠策略

首次测试楼层：512 层
策略序列：512 → 768 → 896 → 960 → 984 → 993 → 997 → 999 → 1000

附录：PPT内容一览

生成的 PPT（玻璃弹珠测试方案.pptx）包含 10 页：

页码	标题	内容
1	封面	标题、副标题、装饰
2	问题描述	场景 + 约束条件
3	核心解题思路	平衡策略、动态规划、二分思想
4	2个弹珠：数学推导	推导步骤 + 45次结论

页码	标题	内容
5	动态规划通用解法	公式 + 示例
6	2个弹珠：测试策略演示	测试序列 + 三种场景
7	多弹珠方案对比	2~10个弹珠对比表
8	5个与10个弹珠方案详解	方案说明 + 数据
9	三种方案测试序列对比	2 vs 5 vs 10 序列
10	总结与思考	核心结论 + 关键数字

温馨提示：

- 视频中的 DP 推演部分，建议配合 PPT 第 5 页一起讲解
- 策略演示可以拿 PPT 第 6 页做动画（碎 vs 没碎的分支）
- 代码部分可以录屏展示实际运行结果
- 总时长控制在 20 分钟左右最佳